Fundamentos de Programación II

Descripción Ver entrega

Límite de entrega: martes, 1 de octubre de 2024, 10:30
Ficheros requeridos: Pruebas.java, InterestGroup.java, ContainerBase.java, Node.java (Descargar)
Tipo de trabajo: Individual

Gestión de Grupos de Interés

Se pide **construir** la clase `InterestGroup` en Java. Cada objeto de dicha clase representa un **grupo** de personas que comparten un cierto interés.

Los miembros de un grupo se almacenarán empleando **lista encadenada**. Para ello, la clase `InterestGroup` habrá de heredar de la clase `ContainerBase`. Esta suministra el atributo `first` de la clase `Node`, que referencia el **inicio** de la lista. Además, define el método `toString()`, lo que nos permite **visualizar** fácilmente los miembros.

Los objetos de la clase `Node` (**véase** su código suministrado) representan los nodos de la lista. Disponen de dos atributos: `value`, de tipo `String` (a usar para almacenar el nombre del **miembro**), y `next`, de tipo `Node` (a usar para referenciar el **siguiente** de la lista).

Para construir la clase `InterestGroup` deberás desarrollar los siguientes métodos (pudiendo añadir otros métodos y atributos que fueran adecuados):

Métodos a desarrollar:

1. **Método** `isMember`
   Habrá de devolver `true` si el nombre pasado por parámetro está en el grupo; `false` en caso contrario.

2. **Método** `addMember`
   Deberá añadir un nuevo miembro (nombre pasado por parámetro) al grupo si no estaba previamente, devolviendo `true` si lo añade, y `false` si no.

3. **método** `getSize`
   Habrá de devolver el número de miembros del grupo.

4. **Método** `union`
   Habrá de devolver un **nuevo** grupo conteniendo tanto los miembros del propio grupo como los del grupo recibido por parámetro, evitando duplicados.

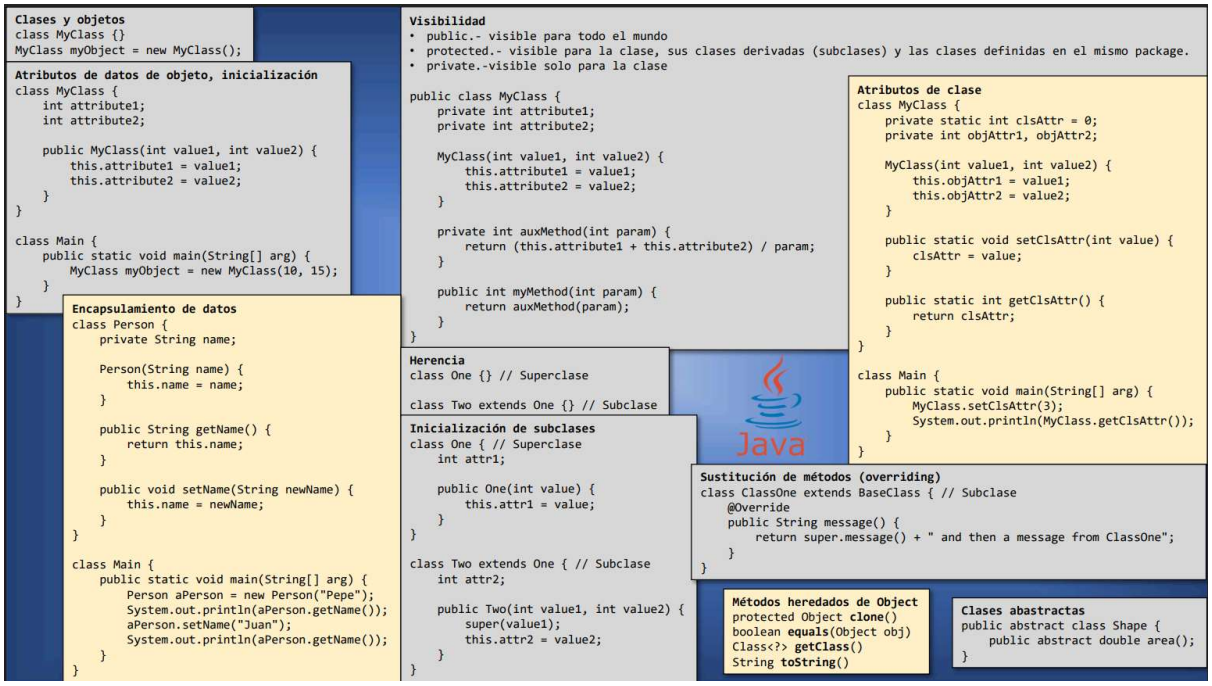
5. **Método** `removeMember`
   Deberá eliminar del grupo, si estuviera, el miembro de nombre recibido por parámetro, devolviendo `true` . De lo contrario, deberá devolver `false`.

Notas y recomendaciones:

- **El informe de evaluación** le **proporciona el código empleado** en cada caso y el error encontrado: error de compilación , fallo en los resultados , tiempo límite superado , etc.
- **Localice los errores** en su código modificando y ejecutando el fichero `Pruebas.java`. Note que **puede copiar y pegar código desde el resultado de la evaluación**. El contenido de este fichero no se evalúa.
- El tipo de prueba de evaluación empleado en este ejercicio puede requerir más de 20 segundos.
- El orden de los miembros en el grupo no es relevante.

Ejemplo de uso:

```java
        InterestGroup group1 = new InterestGroup();
        InterestGroup group2 = new InterestGroup();

        group1.addMember("Ana");
        group1.addMember("Luis");
        group1.addMember("Marta");

        group2.addMember("Luis");
        group2.addMember("Carlos");
        group2.addMember("Elena");

        System.out.println(group1);
        // ➡️ [Marta]->[Luis]->[Ana]->null (El orden no es relavante)

        System.out.println(group1.isMember("Luis")); // true
        System.out.println(group1.getSize()); // 3

        InterestGroup unionGroup = group1.union(group2);
        System.out.println(unionGroup);
        // ➡️ [Elena]->[Carlos]->[Marta]->[Luis]->[Ana]->null (El orden no es relavante)

        unionGroup.remove_member("Elena");
        System.out.println(unionGroup);
        // ➡️ [Carlos]->[Marta]->[Luis]->[Ana]->null
```



Ficheros requeridos

Pruebas.java

```java
public class Pruebas {
    public static void main(String[] args) {
        InterestGroup group1 = new InterestGroup();
        InterestGroup group2 = new InterestGroup();

        System.out.println("Inicialmente:");
        System.out.println(group1); // ➡️ null
        System.out.println(group2); // ➡️ null

        // Añadimos miembros al primer grupo
        group1.addMember("Ana");
        group1.addMember("Luis");
        group1.addMember("Marta");

        System.out.println("\nGrupo 1 tras añadir miembros:");
        System.out.println(group1); // ➡️ [Marta]->[Luis]->[Ana]->null

        // Añadimos miembros al segundo grupo
        group2.addMember("Luis");
        group2.addMember("Carlos");
        group2.addMember("Elena");

        System.out.println("\nGrupo 2 tras añadir miembros:");
        System.out.println(group2); // ➡️ [Elena]->[Carlos]->[Luis]->null

        // Comprobamos si ciertos miembros están en los grupos
        System.out.println("\nComprobación de miembros:");
        System.out.println("Luis en grupo1: " + group1.isMember("Luis")); // true
        System.out.println("Carlos en grupo1: " + group1.isMember("Carlos")); // false
        System.out.println("Elena en grupo2: " + group2.isMember("Elena")); // true

        // Tamaño de los grupos
        System.out.println("\nTamaño de los grupos:");
        System.out.println("Grupo1: " + group1.getSize()); // 3
        System.out.println("Grupo2: " + group2.getSize()); // 3

        // Unión de grupos
        InterestGroup unionGroup = group1.union(group2);
        System.out.println("\nUnión de grupo1 y grupo2:");
        System.out.println(unionGroup); // ➡️ [Ana]->[Luis]->[Marta]->[Carlos]->[Elena]->null

        // Probar que no se agregan duplicados
        group1.addMember("Luis"); // Ya está
        System.out.println("\nGrupo1 tras intentar añadir a 'Luis' otra vez:");
        System.out.println(group1); // ➡️ [Ana]->[Luis]->[Marta]->null

        // Probar que se eliminan miembros
        group1.removeMember("Marta");
        System.out.println("\nGrupo1 tras eliminar 'Marta':");
        System.out.println(group1); // ➡️ [Ana]->[Luis]->null
    }
}

```

InterestGroup.java

```java
// Desarrolle aquí su código

```

ContainerBase.java

```java
public class ContainerBase{
    protected Node first;
    public String toString() {
        // No se muestra
    }
}

```

Node.java

```java
public class Node{
    public String value;
    public Node next;
    public Node(String value){
        this.value = value;
    }
}

```